

Symbolic-Permutation-Based Direct Sparse Jacobian Assembly for Accelerated Power-Flow Analysis

Tianshi Cheng, *Member, IEEE*, and Venkata Dinavahi, *Fellow, IEEE*

Abstract—The repeated assembly of the sparse Jacobian matrix constitutes a primary computational bottleneck in the Newton–Raphson power-flow analysis. Through an algebraic equivariance analysis of the Jacobian matrix, this paper proposes a direct sparse Jacobian assembly method based on symbolic permutation. By pre-applying the permutation to the admittance matrix during a one-time symbolic phase, the runtime bus-type projection overhead is entirely eliminated. Consequently, complex Jacobian differentiation and global matrix assembly fuse into a single operation, yielding a single-pass, branchless algorithm that streams values directly into contiguous memory. The framework is realized in the open-source `RustPower` solver. Benchmarks on IEEE test systems and PEGASE 9241 demonstrate that the proposed assembly is $16\text{--}21\times$ faster than the MATPOWER-style baseline and $2.8\text{--}7.2\times$ faster than the Schäfer–Braun optimized assembly. End-to-end, the overall solve time is $1.7\text{--}3.5\times$ faster than the native C++ `LightSim2Grid`, while achieving a 15-fold reduction in peak memory allocation and leaving numerical results and convergence behavior unchanged. The method extends directly to general problems where a sparse Jacobian inherits its structure from a static network operator, including transient stability analysis, EMT simulation, and OPF Hessian assembly.

Index Terms—Power flow, Newton–Raphson method, Jacobian matrix, sparse matrices, symbolic analysis, compressed sparse column, data-oriented design, high-performance computing.

NOMENCLATURE

n, b	Number of buses; number of branches.
$\mathbf{A}$	Branch–bus incidence matrix.
$\mathbf{Y}_{br}, \mathbf{Y}_{bus}$	Primitive branch admittance; nodal admittance matrix.
$\mathbf{V}, \mathbf{I}_{bus}$	Complex bus voltage and injected-current vectors.
$ \mathbf{V} , \boldsymbol{\theta}$	Bus voltage magnitude and angle vectors.
$\hat{\mathbf{V}}$	Unit-modulus voltage, $\hat{\mathbf{V}} = e^{j\boldsymbol{\theta}}$.
$\mathbf{S}_{bus}$	Complex bus power injection, $\mathbf{P} + j\mathbf{Q}$.
$\mathbf{S}_{bus}^{sp}$	Specified (scheduled) bus power injection.
$\mathbf{x}, \mathbf{x}_{full}$	State vector of unknowns; full $2n$ state.
$\mathbf{F}$	Power-mismatch (residual) vector.
ν	Newton–Raphson iteration index.
$\mathbf{J}_{full}, \mathbf{J}_{red}$	Full and bus-type-reduced power-flow Jacobian.
$\mathcal{P}$	Reduction slicer $[\mathbf{I}_{N_{red}} \mid \mathbf{0}]$.
$\mathbf{T}$	Orthogonal bus-type permutation matrix.
$\text{struct}(\cdot)$	Boolean (0/1) sparsity matrix of a matrix.
B_Y, B_J	Boolean sparsity matrices of $\mathbf{Y}_{bus}$ and $\mathbf{J}_{red}$.
N_{pv}, N_{pq}	Number of PV and PQ buses.
N_{red}	$N_{pv} + 2N_{pq}$, the reduced-system dimension.
NR	Newton–Raphson.
CSC, CSR	Compressed sparse column / row storage.
ECS	Entity–component–system.

I. INTRODUCTION

This work was supported by the Natural Sciences and Engineering Research Council of Canada (NSERC) and Mitacs.

T. Cheng is with the RTDS Technologies Inc., Winnipeg, MB R3T 2E1, Canada, and V. Dinavahi is with the Department of Electrical and Computer Engineering, University of Alberta, Edmonton, AB T6G 2V4, Canada (e-mail: tcheng1@ualberta.ca; dinavahi@ualberta.ca).

STEADY power-flow analysis is the computational kernel beneath planning, operations, contingency screening, and optimization of electric power systems. The Newton–Raphson (NR) method has dominated this task since its introduction by Tinney and Hart in 1967 [1], together with optimally ordered sparse factorization [2] establishing the direct-solver paradigm that remains the standard today. At each Newton step, the Jacobian matrix is assembled and the resulting sparse linear system is solved by direct methods such as KLU [3], which separate a one-time symbolic analysis from repeated numeric re-factorization, making the latter an allocation-free, cache-efficient pipeline. For extremely large or ill-conditioned systems where the Jacobian condition number degrades and direct factorization becomes numerically unstable, the inner linear system may instead be solved by preconditioned GMRES or BiCGStab [4], [5]; however, the pre-conditioner still requires the explicit Jacobian structure, and for standard benchmarks these Newton–Krylov variants remain slower than a well-tuned direct solver, leaving LU-based NR as the workhorse in practice. Robustness extensions—Iwamoto–Tamura damping [6], Levenberg–Marquardt regularization [7], [8], and continuous Newton methods [9], [10]—addressed convergence fragility at the cost of more outer iterations and hence more frequent Jacobian evaluations.

Consequently, the assembly of the Jacobian matrix—an indispensable step in every Newton-based iteration—has superseded sparse factorization as the dominant computational cost. Classical textbooks often formulated the Jacobian by element-wise differentiation of the scalar polar power-flow equations, incurring $\Theta(nnz)$ transcendental function evaluations and tedious index bookkeeping for each entry [11]. Zimmerman et al. [12] revolutionized this practice by maintaining the algebraic integrity of the network entirely in the complex domain through Wirtinger Calculus. The resulting matrix-differential formulation collapses the transcendental burden from a traditional $\Theta(nnz)$ element-wise sweep to a single $\Theta(n)$ vector map per iteration, and it became the foundational paradigm for modern power-flow software. This complex-domain approach was ported to Python as PYPower [13], extended into an object-oriented modeling environment in pandapower [14], and reimplemented with a C++/Eigen backend in LightSim2Grid [15] for native speed.

In 2018, Schäfer and Braun [16] were among the first to systematically identify the Jacobian-assembly bottleneck, replacing MATPOWER’s block-stacking pipeline with an optimized CSR formation fused by Numba JIT compilation, reporting $3\text{--}14\times$ assembly speedups compared to MATPOWER. The C++ LightSim2Grid [15] also adopted a similar element-level assembly approach. Subsequent work has pushed the parallel

frontier: heterogeneous CPU–GPU solvers exploit invariant sparsity across ensemble members to amortize symbolic factorization over thousands of N–1 or Monte-Carlo cases [17]–[19] as well as large-scale optimization problems [20].

Yet all of these works and advances—whether in MATLAB, Python, C++, or CUDA—share a common architectural pitfall: they remain trapped in the three-step “differentiate, partition, assemble” pipeline of Jacobian assembly for Newton iteration methods. Differentiation evaluates full $2n \times 2n$ complex sensitivities with thousands of unused computations of PV and slack buses; partitioning scatters entries through pointer-chasing and dynamic indexing; assembly allocates temporary blocks and copies them into the global Jacobian. These structural overheads dominate the hot loop—81–91% of per-iteration time (Fig. 3)—and they seem to be *fundamental*: no engineering tuning can eliminate the three steps in the current open-source implementations.

As shown in Fig. 1, this paper proposes a symbolic-permutation-based direct sparse Jacobian assembly method that significantly reduces the iteration structural overhead in Newton–Raphson power-flow analysis. The work consists of three main contributions:

- 1) **Permutation-equivariance theorem.** Theorem 1 proves that the Jacobian assembly commutes with orthogonal bus permutation, identifying the bus-type projection as the algebraic root cause of block-assembly fragmentation and enabling its hoisting to a one-time symbolic phase (Section III).
- 2) **Single-pass direct-fill algorithm.** With the Jacobian structure fixed by Corollaries 1–2, per-iteration assembly reduces to a branch-free, allocation-free $\Theta(\text{nnz})$ sweep: each column of the admittance matrix is differentiated directly into its final position in the reduced Jacobian, eliminating intermediate sensitivity blocks, per-iteration sparse structure reconstruction, and wasted derivative evaluations for dropped states (Section III–Section IV).
- 3) **Open-source RustPower implementation.** The method (noted as V2) is implemented in `RustPower`, an open-source data-oriented ECS framework. Isolated assembly benchmarks demonstrate $16\times$ – $21\times$ speedup over baseline MATPOWER-style [12] formulations (V0) and $2.8\times$ – $7.2\times$ over the optimized pandapower-style algorithm [16] (V1), eliminating 93.8%–95.3% of the baseline assembly overhead (and up to 86.2% of the residual overhead in V1). End-to-end comparison against native C++ `LightSim2Grid` and `pandapower` confirms $1.7\times$ – $3.5\times$ overall solve speedup at machine-precision accuracy, with a 15-fold reduction in peak memory allocation (Section V).

Section VI concludes the paper and discusses future extensions to transient stability analysis, EMT simulation, and OPF Hessian assembly.

II. MATHEMATICAL FORMULATION

A. Network and Admittance Matrix

In steady-state analysis, a power network of n buses and b branches is described by its branch–bus incidence matrix

$\mathbf{A}$; where $\mathbf{Y}_{\text{br}} = \text{blkdiag}(y_1, \dots, y_b)$ is the branch admittance matrix, the nodal admittance matrix can be expressed as

$$\mathbf{Y}_{\text{bus}} = \mathbf{A}^\top \mathbf{Y}_{\text{br}} \mathbf{A}. \quad (1)$$

The network matrix $\mathbf{Y}_{\text{bus}}$ fundamentally inherits the properties of a weighted Laplacian matrix.

Buses are ordered as [PQ | PV | slack], by the number of state variables each bus contributes: PQ buses carry both θ and $|V|$ as states and lead, PV buses carry θ only and follow, slack buses carry no state and close the list. The “missing state” types therefore concentrate on the right. The ordering is realised by a one-time permutation $\mathbf{T}$ applied before iteration; the pre-ordered admittance matrix $\mathbf{T}\mathbf{Y}_{\text{bus}}\mathbf{T}^\top$ locks the sparse topology, and every subsequent block split coincides with a contiguous row range.

B. The Power-Flow Equation

The complex bus power injection is the nonlinear function of the bus voltage given by

$$\mathbf{S}_{\text{bus}}(\mathbf{V}) = \text{diag}(\mathbf{V}) \overline{\mathbf{Y}_{\text{bus}} \mathbf{V}}, \quad (2)$$

with $\overline{(\cdot)}$ complex conjugation. Expanded element-wise, (2) becomes the classical polar form

$$P_i = v_i \sum_{k \sim i} v_k (G_{ik} \cos \theta_{ik} + B_{ik} \sin \theta_{ik}), \quad (3)$$

$$Q_i = v_i \sum_{k \sim i} v_k (G_{ik} \sin \theta_{ik} - B_{ik} \cos \theta_{ik}), \quad (4)$$

where $\theta_{ik} = \theta_i - \theta_k$, $G_{ik} + jB_{ik}$ denotes the (i, k) -th element of the admittance matrix $\mathbf{Y}_{\text{bus}}$, and the summation runs over all buses k connected to bus i .

Although these scalar formulations are common in textbooks to directly differentiate and build the Jacobian, this representation is highly inefficient. Every individual entry in the resulting Jacobian inherits distinct transcendental pairs $(\sin \theta_{ik}, \cos \theta_{ik})$, forcing conventional element-wise differentiation to be a slow $\Theta(\text{nnz})$ operation per iteration.

This heavy algebraic penalty historically obscured the potential of efficient direct Jacobian assembly. Consequently, modern tools adopted complex matrix differentiation to bypass the high transcendental function cost.

C. Analytical Matrix Differentiation and Jacobian Assembly

The Newton–Raphson method eliminates the non-linear power mismatch by iteratively solving the linearized system

$$\mathbf{J}^{(\nu)} \Delta \mathbf{x}^{(\nu)} = -\mathbf{F}(\mathbf{x}^{(\nu)}), \quad \mathbf{x}^{(\nu+1)} = \mathbf{x}^{(\nu)} + \Delta \mathbf{x}^{(\nu)}, \quad (5)$$

where ν is the iteration counter. In this holistic formulation, the global residual vector $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ is directly mapped from the complex algebraic network constraint

$$\mathbf{F}(\mathbf{V}) = \mathbf{S}_{\text{bus}}^{\text{sp}} - \text{diag}(\mathbf{V}) \overline{\mathbf{Y}_{\text{bus}} \mathbf{V}} = \mathbf{0}, \quad (6)$$

and the Jacobian is expressed in matrix form as

$$\mathbf{J}_{\text{full}} = [\partial \mathbf{S}_{\text{bus}} / \partial \boldsymbol{\theta} \quad \partial \mathbf{S}_{\text{bus}} / \partial |\mathbf{V}|], \quad (7)$$

where $\mathbf{S}_{\text{bus}}^{\text{sp}}$ contains the constant constraints (full $\mathbf{S}$ for PQ buses, $\mathbf{P}$ only for PV buses), and $\mathbf{V}$ has constant

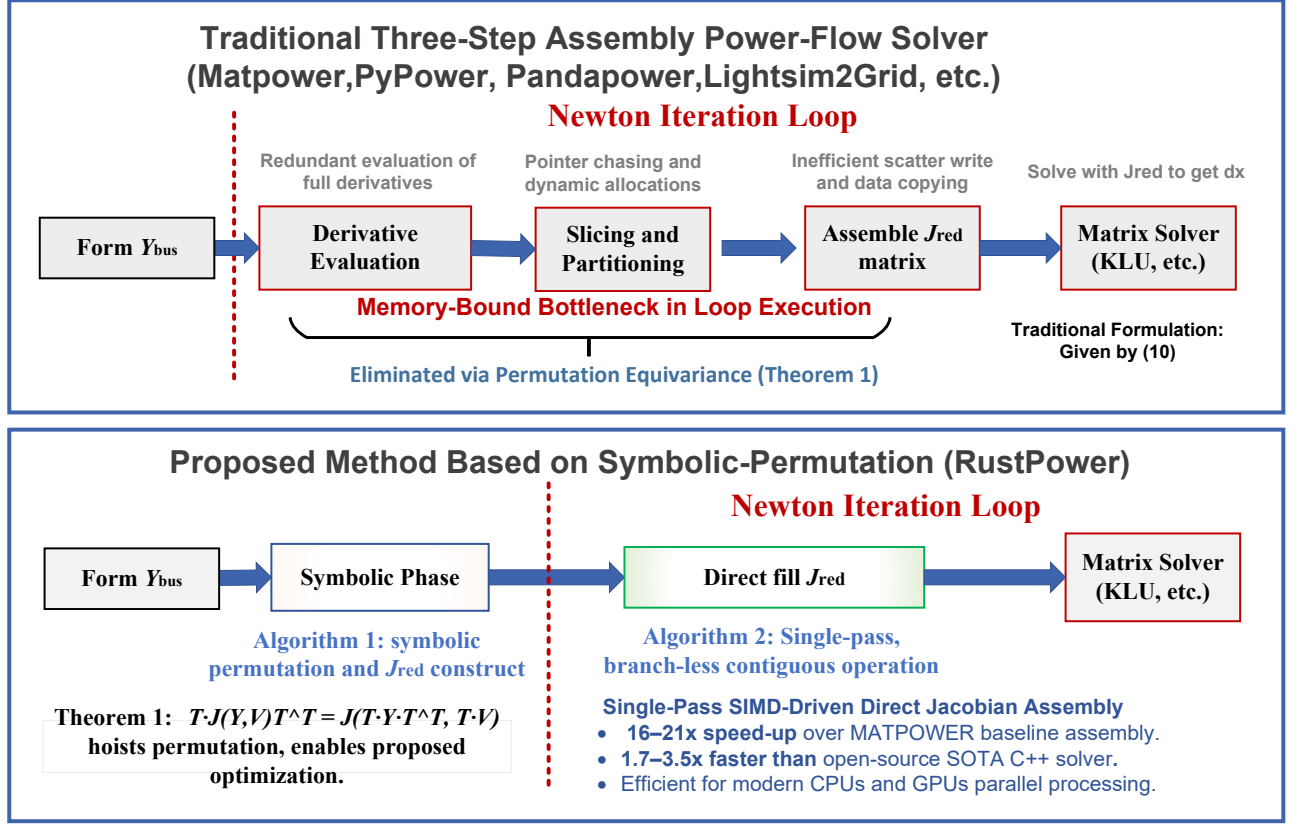


Fig. 1. Overview of the proposed method's core innovation. By hoisting topology-dependent permutations offline via Theorem 1, the proposed framework eliminates runtime slicing and scattered writes, condensing the traditional loop into a single-pass direct-fill stream to achieve up to $21\times$ speedup and outperform the state-of-the-art open-source C++ solver.

constraints ($|V|$ for PV, $V \angle \theta$ for slack buses). In their pioneering framework, Zimmerman et al. [12] introduced an inspiring, mathematically elegant matrix-differential method for MATPOWER. Instead of expanding the governing power equations into scalar polar loops prematurely, they maintained the algebraic integrity of the network entirely within the complex domain through the continuous exponential voltage alignment $V = \text{diag}(|V|)\hat{V}$ with $\hat{V} = e^{j\theta}$. By exploiting the compact network current dependency $I_{bus} = Y_{bus}V$, this matrix-differential formulation directly yields the exact complex sensitivity operators:

$$\frac{\partial S}{\partial \theta} = j \text{diag}(V) (\text{diag}(\overline{I_{bus}}) - \overline{Y_{bus}} \text{diag}(\overline{V})), \quad (8)$$

$$\frac{\partial S}{\partial |V|} = \text{diag}(V) \overline{Y_{bus}} \text{diag}(\hat{V}) + \text{diag}(\overline{I_{bus}}) \text{diag}(\hat{V}). \quad (9)$$

This holistic complex-domain formulation entirely eliminates the heavy trigonometric evaluations from the differentiation phase, successfully collapsing the transcendental computational burden from a traditional $\Theta(\text{nnz})$ down to a single node-level $\Theta(n)$ vector map per iteration.

To mitigate the heavy computational burden of sparse matrix-vector products in conventional complex-domain formulations, Schäfer and Braun [16] brilliantly deployed Numba JIT compilation in pandapower to fuse these underlying oper-

ations. By mapping elements directly based on the structural invariance of the network topology, their approach successfully streamlined derivative evaluations and bypassed some of the intermediate matrix products, delivering an impressive $3\times$ – $14\times$ speedup.

However, while this element-wise fusion successfully accelerated the differentiation phase, the underlying solver remains trapped in a classic architectural pitfall. Because the complex power-flow equations are non-holomorphic, conventional implementations must maintain extensive layout-mapping tables to partition derivatives into discrete J_{11} through J_{22} sub-blocks. This structural dependency burdens the hot loop with continuous pointer-dereferencing and non-contiguous scatter writes that completely disrupt hardware prefetching.

To resolve this architectural pitfall, the power-flow equations demand further mathematical analysis.

III. SYMBOLIC MATRIX ANALYSIS OF DOMAIN PROJECTION AND REORDERING

A. Non-Holomorphic Deadlock and the Dual Real-Mapping Paradigm

The fundamental mathematical bottleneck of conventional power-flow algorithms originates from a non-holomorphic deadlock in the complex domain. The governing complex

power injection equation $\mathbf{S}_{\text{bus}}(\mathbf{V}) = \text{diag}(\mathbf{V})\overline{\mathbf{Y}_{\text{bus}}\mathbf{V}}$ violates the Cauchy-Riemann conditions due to the non-differentiable nature of the conjugation operator, forcing a paradigm shift that maps the complex sensitivities onto a dual real-coordinate system

$$\begin{aligned} \mathbf{J}_{\text{red}} &= \mathcal{P} (\mathbf{T}_{\text{block}} \mathbf{J}_{\text{full}}(\mathbf{Y}_{\text{bus}}, \mathbf{V}) \mathbf{T}_{\text{block}}^\top) \mathcal{P}^\top \\ &= \mathcal{P} \begin{bmatrix} \mathbf{T} & \mathbf{0} \\ \mathbf{0} & \mathbf{T} \end{bmatrix} \begin{bmatrix} \Re \left(\frac{\partial \mathbf{S}}{\partial \theta} \right) & \Re \left(\frac{\partial \mathbf{S}}{\partial |\mathbf{V}|} \right) \\ \Im \left(\frac{\partial \mathbf{S}}{\partial \theta} \right) & \Im \left(\frac{\partial \mathbf{S}}{\partial |\mathbf{V}|} \right) \end{bmatrix} \begin{bmatrix} \mathbf{T}^\top & \mathbf{0} \\ \mathbf{0} & \mathbf{T}^\top \end{bmatrix} \mathcal{P}^\top. \end{aligned} \quad (10)$$

Under this traditional real-mapping paradigm, contemporary implementations—including state-of-the-art automatic differentiation (AutoDiff) frameworks—are bound by a conventional core assumption: that the unconstrained global matrix $\mathbf{J}_{\text{full}}$ must be explicitly materialized prior to handling any spatial configurations. Consequently, traditional power-flow solvers and AutoDiff engines are restricted to evaluating the full, unconstrained matrix, remaining fundamentally agnostic to external boundary constraints and spatial node partitions, which inherently forces conventional architectures to execute three computationally catastrophic operations inside every hot-loop iteration:

- 1) **Redundant Differentiation:** Wasting significant CPU cycles and arithmetic throughput evaluating derivatives for dropped states (e.g., slack and PV bus voltage magnitudes) that are subsequently discarded.
- 2) **Indirect Spatial Partitioning:** Relying on extensive layout-mapping tables to index and route derivatives into traditional $\mathbf{J}_{11}$ through $\mathbf{J}_{22}$ sub-blocks, introducing severe pointer-dereferencing and indirection overheads inside the hot loop.
- 3) **Non-Contiguous Scatter Assembly:** Enforcing dimensional reduction via scattered writes that distribute values across disjoint matrix segments, which completely disrupts hardware prefetching, invalidates CPU cache lines, and severely limits parallel scalability.

These limitations transform the iteration hot path into a performance graveyard of branch mispredictions and cache invalidations, which also significantly lower the parallel efficiency on modern CPUs and GPUs.

B. Permutation Equivariance of the Sensitivity

The complex sensitivities (8)–(9) are built from three primitive operations on $\mathbf{Y}_{\text{bus}}$ and $\mathbf{V}$: diagonal scaling, matrix product, and complex conjugation. Each of them commutes with conjugation by an orthogonal permutation. The elementary fact at the bottom of the proof is the following

Lemma 1 (Diagonal-permutation commutation). *For any vector $\mathbf{x}$ and orthogonal permutation matrix $\mathbf{T}$ with $\mathbf{T}\mathbf{T}^\top = \mathbf{I}$,*

$$\mathbf{T} \text{diag}(\mathbf{x}) \mathbf{T}^\top = \text{diag}(\mathbf{T}\mathbf{x}). \quad (11)$$

Proof. Let π be the permutation with $T_{ij} = \delta_{i,\pi(j)}$. Then

$$\begin{aligned} (\mathbf{T} \text{diag}(\mathbf{x}) \mathbf{T}^\top)_{ij} &= \sum_k \delta_{i,\pi(k)} \delta_{j,\pi(k)} x_k = \delta_{ij} x_{\pi^{-1}(i)} \\ &= (\text{diag}(\mathbf{T}\mathbf{x}))_{ij}. \end{aligned} \quad (12)$$

Theorem 1 (Permutation equivariance of the Jacobian). *Let $J(\mathbf{Y}, \mathbf{V})$ denote either complex sensitivity matrix of (8)–(9). For any orthogonal permutation $\mathbf{T} \in \{0, 1\}^{n \times n}$,*

$$\mathbf{T} J(\mathbf{Y}, \mathbf{V}) \mathbf{T}^\top = J(\mathbf{T}\mathbf{Y}\mathbf{T}^\top, \mathbf{T}\mathbf{V}). \quad (13)$$

Proof. Each primitive operation in J commutes with $\mathbf{T}$ -conjugation:

- matrix product, $\mathbf{T} \mathbf{A} \mathbf{B} \mathbf{T}^\top = (\mathbf{T} \mathbf{A} \mathbf{T}^\top)(\mathbf{T} \mathbf{B} \mathbf{T}^\top)$ since $\mathbf{T}^\top \mathbf{T} = \mathbf{I}$;
- diagonalisation, $\mathbf{T} \text{diag}(\mathbf{x}) \mathbf{T}^\top = \text{diag}(\mathbf{T}\mathbf{x})$ by Lemma 1;
- complex conjugation, $\mathbf{T} \overline{\mathbf{A}} \mathbf{T}^\top = \overline{\mathbf{T} \mathbf{A} \mathbf{T}^\top}$ since $\mathbf{T}$ is real.

Apply these to (8)–(9) entrywise, using $\mathbf{T} \mathbf{I}_{\text{bus}}(\mathbf{Y}, \mathbf{V}) = (\mathbf{T}\mathbf{Y}\mathbf{T}^\top)(\mathbf{T}\mathbf{V}) = \mathbf{I}_{\text{bus}}(\mathbf{T}\mathbf{Y}\mathbf{T}^\top, \mathbf{T}\mathbf{V})$. $\square$

$\mathbf{J}_{\text{full}}$ inherits this equivariance because the transformation $\mathbf{T}$ is real. We select a state-level permutation that concentrates all active variables $[\theta_{\text{PQ}}; \theta_{\text{PV}}; |\mathbf{V}|_{\text{PQ}}]$ at the front, while trailing the omitted PV and slack states at the end. The reduction operator simplifies to a trivial memory slicer, completely eliminating runtime coordinate projections.

Corollary 1 (Pre-permutation absorbs the projection). *Let $\mathcal{P} = [\mathbf{I}_a \ \mathbf{0}]$ denote the static truncation matrix. The row-and-column permutation $\mathbf{T}$ isolates the active variables, partitioning the transformed Jacobian into a block layout:*

$$\mathbf{T} \mathbf{J}_{\text{full}} \mathbf{T}^\top = \begin{bmatrix} \mathbf{J}_{aa} & \mathbf{J}_{ad} \\ \mathbf{J}_{da} & \mathbf{J}_{dd} \end{bmatrix},$$

where a and d denote the active and dropped domains. Because the block structuring forces the projection to collapse into identity matrices, the truncation operator $\mathcal{P}$ directly absorbs the transformation. Combining this layout with Theorem 1 yields:

$$\mathbf{J}_{\text{red}} = \mathcal{P} \mathbf{J}_{\text{full}}(\mathbf{T}\mathbf{Y}_{\text{bus}} \mathbf{T}^\top, \mathbf{T}\mathbf{V}) \mathcal{P}^\top \equiv \mathbf{J}_{aa}. \quad (14)$$

The bus permutation $\mathbf{T}$ is applied once before execution. Due to this block alignment, the runtime projection is entirely absorbed into the source matrix, leaving the solver to extract the top-left block via a single memory slice.

Corollary 2 (Boolean structural mapping). *Apply the structural operator $\text{struct}(\cdot)$ to (14). Because the projection $\mathcal{P}$ acts as a binary slicer, the operator passes through it:*

$$\text{struct}(\mathbf{J}_{\text{red}}) = \mathcal{P} \text{struct}(\mathbf{J}_{\text{full}}(\mathbf{T}\mathbf{Y}_{\text{bus}} \mathbf{T}^\top, \mathbf{T}\mathbf{V})) \mathcal{P}^\top. \quad (15)$$

With $B_Y = \text{struct}(\mathbf{Y}_{\text{bus}})$, the real-linear split duplicates the pattern into four quadrant sub-blocks:

$$B_{\mathbf{J}_{\text{full}}} = \text{struct}(\mathbf{J}_{\text{full}}) = \begin{bmatrix} B_Y & B_Y \\ B_Y & B_Y \end{bmatrix} \in \{0, 1\}^{2n \times 2n}, \quad (16)$$

and the equivariance carries this pattern through to the permuted matrix, $\text{struct}(\mathbf{J}_{\text{full}}(\mathbf{T}\mathbf{Y}_{\text{bus}} \mathbf{T}^\top, \mathbf{T}\mathbf{V})) = \mathbf{T} B_{\mathbf{J}_{\text{full}}} \mathbf{T}^\top$. Hence,

$$\text{struct}(\mathbf{J}_{\text{red}}) = \mathcal{P} \mathbf{T} B_{\mathbf{J}_{\text{full}}} \mathbf{T}^\top \mathcal{P}^\top. \quad (17)$$

Fig. 2 shows the IEEE 39-bus system; the matrix patterns confirm Theorem 1 and Corollaries 1–2.

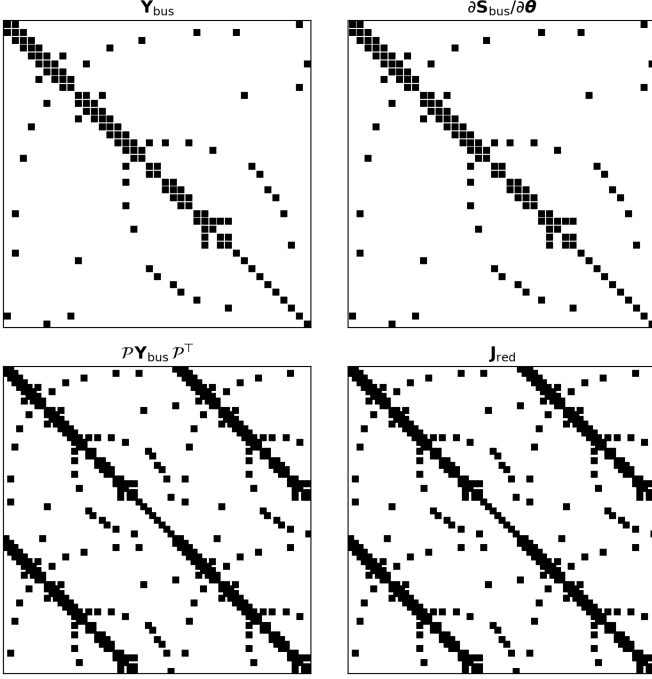


Fig. 2. Permutation equivariance and reduction on IEEE 39, buses in [PQ | PV | slack] order. *Top*: the complex sensitivity $\partial \mathbf{S}_{\text{bus}} / \partial \theta$ inherits $\mathbf{Y}_{\text{bus}}$'s nonzero positions (Theorem 1; $\partial \mathbf{S}_{\text{bus}} / \partial |\mathbf{V}|$ is identical, omitted for space). *Bottom*: $\mathbf{J}_{\text{red}}$ is the top-left $(N_{\text{pv}} + 2N_{\text{pq}})$ -square block of the full Jacobian assembled on the pre-permuted admittance matrix $\mathbf{T}\mathbf{Y}_{\text{bus}}\mathbf{T}^T$ (Corollary 2).

C. Direct Closed-Form Address Mapping

By expanding the analytical complex sensitivities (8)–(9) along individual columns, the exact structural column locality evaluates by inspection:

$$\begin{aligned} \left[\frac{\partial \mathbf{S}_{\text{bus}}}{\partial \theta} \right]_{:,k} &= j \operatorname{diag}(\mathbf{V}) (\overline{I_k} e_k - \overline{V_k} [\mathbf{Y}_{\text{bus}}]_{:,k}), \\ \left[\frac{\partial \mathbf{S}_{\text{bus}}}{\partial |\mathbf{V}|} \right]_{:,k} &= \overline{V_k} \operatorname{diag}(\mathbf{V}) [\mathbf{Y}_{\text{bus}}]_{:,k} + \hat{V}_k \overline{I_k} e_k. \end{aligned} \quad (18)$$

where e_k is the k -th unit vector. The equations establish that the numerical derivation of the k -th column of the operational Jacobian requires the same k -th column of $\mathbf{Y}_{\text{bus}}$, leading to direct address mapping instead of generic sparse matrix indices.

Since (18) executes strictly within decoupled column scopes, the nonzero destination addresses inside the real operational Jacobian $\mathbf{J}_{\text{red}}$ are computed directly from the static topology $\operatorname{struct}(\mathbf{Y}_{\text{bus}})$ without matrix traversal. For any nonzero branch $(i, k) \in \operatorname{struct}(\mathbf{Y}_{\text{bus}})$, its pre-allocated NNZ memory locations can be evaluated via a unified linear step stride

Phase 1 (θ -state column variations, $k \in [0, N_{\text{pv}} + N_{\text{pq}} - 1]$):

$$\sigma_\theta(i, k) = c_{\theta k} + \mathbb{I}(i \in \mathcal{I}_{\text{PQ}}) \cdot a_k + \rho(i, k) \quad (19)$$

Phase 2 ($|\mathbf{V}|$ -state column variations, $k \in [0, N_{\text{pq}} - 1]$):

$$\sigma_{|\mathbf{V}|}(i, k) = c_{|\mathbf{V}|k} + \mathbb{I}(i \in \mathcal{I}_{\text{PQ}}) \cdot a_k + \rho(i, k) \quad (20)$$

The exact memory coordinates for all four implicit Jacobian sub-blocks are dynamically unified by three static topological attributes: the base column pointer $c_{(\cdot)}$, the zero-based row

offset $\rho(i, k)$ inside the current column slice, and the isolated active nonzero cardinality a_k . The architectural and algebraic superiorities of this mapping are summarized as follows (Table I):

Single-Pass Contiguous Writes: The completeness ordering T isolates all boundary-constrained dropped nodes into the matrix lower-right corner. Consequently, the conditional indicator $\mathbb{I}(\cdot)$ translates into a monotonically increasing linear offset, forcing the hardware execution thread to automatically skip dropped PV and slack constraints while maintaining uninterrupted single-pass 1D streaming writes.

Layout Generality: Although this single-pass streaming kernel is presented column-wise under the Compressed Sparse Column (CSC) format, the underlying memory dispatch logic generalizes symmetrically to Compressed Sparse Row (CSR) layouts. The global target memory addresses remain entirely predictable a priori across rows, guaranteeing maximum execution throughput on modern SIMD-driven processors.

TABLE I
CONSEQUENCES OF PROJECTION PLACEMENT.

	Conventional / [16]	This work
Placement of T	$\mathbf{J}_{\text{full}}$	$\mathbf{Y}_{\text{bus}}$
$T(\cdot)T^T$ in loop	every iteration	never
Matrix construction	block assembly	direct contiguous write
Write pattern	scattered	sequential, contiguous
Storage order	CSR	CSC

D. The Numeric Fill Operator

With the pattern fixed by (17), the numeric phase only writes values from (8)–(9) to the matrix. By applying the general evaluation operators of those formulas uniformly to every entry, including the diagonal, we obtain the intermediate matrices $\mathcal{U}_\theta$ and $\mathcal{U}_v$. The $\operatorname{diag}(\overline{I}_{\text{bus}})$ terms are strictly diagonal; thus, $\mathcal{U}_\theta$ and $\mathcal{U}_v$ coincide with the exact derivatives off the diagonal, while the diagonal entries can be independently corrected via localized linear patches based on

$$\frac{\partial \mathbf{S}_{\text{bus}}}{\partial \theta} = \mathcal{U}_\theta + j \operatorname{diag}(\mathbf{S}_{\text{bus}}^{(\nu)}), \quad \frac{\partial \mathbf{S}_{\text{bus}}}{\partial |\mathbf{V}|} = \mathcal{U}_v + \operatorname{diag}!(\mathbf{S}_{\text{bus}}^{(\nu)}! \oslash |\mathbf{V}^{(\nu)}|). \quad (21)$$

Per-iteration assembly is therefore reduced to a branch-free $\Theta(\text{nnz})$ sweep—evaluating a uniform relation for every nonzero entry without any $i \stackrel{?}{=} k$ tests—followed by a $\Theta(n)$ constant-time diagonal modification.

IV. IMPLEMENTATION IN RUSTPOWER

A. Data-Oriented Architecture

`RustPower` realizes the proposed method in Rust [21] on a data-oriented ECS architecture [22], [23], a design carried over from the prior real-time ECSGrid work [24]. All power system entities are stored in statically typed, columnar arrays in ECS archetype tables, where archetypes are similar to *Bus* or *Line* tables in `pandapower`. Networks are read in the `pandapower` JSON/CSV format for reproducibility, and the linear solve is delegated, behind a single `Solve` trait, to an interchangeable backend such as `KLU` [3]. The proposed algorithm is not tied to `RustPower` or Rust and is applicable to other implementations without significant modifications.

B. Sparse Storage and Contiguity of Assembly

The column-decoupled nature of (18) aligns natively with the Compressed Sparse Column (CSC) layout. By eliminating intermediate representations, column k of $\mathbf{Y}_{\text{bus}}$ is streamed as a contiguous memory block to directly populate the corresponding columns of $\mathbf{J}_{\text{red}}$ in a single pass. This strictly sequential data flow guarantees optimal spatial locality, enabling the hardware pre-fetcher to function ideally and effectively eliminating cache steering bottlenecks [25].

While the underlying mapping logic can be applied symmetrically to Compressed Sparse Row (CSR) layouts, state-of-the-art sparse direct solvers like KLU [3] inherently execute factorization column-wise and strictly mandate CSC inputs. To exploit this architectural alignment, *RustPower* maintains a unified CSC pipeline across the entire simulation—from the initial network admittance matrix, through the symbolic structure mapping, to the final linear solver backend. This continuous format consistency completely circumvents expensive matrix transposition overheads within the hot loop.

C. The Symbolic Phase

Algorithm 1 formalizes this one-time symbolic pass, tracing the exact coordinate and offset allocation required to construct the locked sparsity pattern of $\mathbf{J}_{\text{red}}$.

The execution progresses sequentially through the columns of the static network admittance matrix $\mathbf{Y}_{\text{bus}}$. For each active column k (Line 1), a localized binary search is performed on its sorted row indices (Line 2). Leveraging the block-ordered node layout [PQ | PV | slack], this bisection splits the column into active and non-active sub-segments, recording the exact boundaries ($pq_end[k]$, $active_end[k]$) and caching the absolute memory index of the diagonal entry ($diag[k]$).

Following the bisection, the active row indices bounded within $[0, n_a]$ are appended directly to the global value layout of $\mathbf{J}_{\text{red}}$ to instantiate the phase-state (θ_k) column (Line 3). Because the pre-permuted $P\mathbf{Y}_{\text{bus}}P^\top$ topology guarantees that row indices are strictly monotonic, this appending operation translates to a zero-branch memory copy of the active slice, during which the structural boundary offsets for the J_{11} and J_{21} sub-blocks are locked. If the current node satisfies the PQ criteria ($k < N_{pq}$), this identical row structure is mirrored sequentially to establish the voltage-magnitude ($|V|_k$) column, capturing the corresponding J_{12} and J_{22} offsets (Line 4). Finally, the column structural configuration is closed into the sparse column pointers (Line 5). The resulting static metadata functions as a permanent *symbolic cache*, leaving the subsequent numeric phase entirely free from runtime block-stacking or index-shifting computations.

D. The Direct Single-Pass Numeric Phase

The per-iteration assembly kernel, formalized as FILL (Algorithm 2), executes as a branch-free, streaming numeric pipeline to fill $\mathbf{J}_{\text{red}}$ directly. By utilizing the pre-computed metadata from the symbolic pass (Algorithm 1), the numeric kernel completely eliminates runtime memory allocation and spatial coordinate partitioning. Leveraging the permutation

Algorithm 1 Symbolic phase — build CSC pattern of $\mathbf{J}_{\text{red}}$

Require: $\mathbf{Y}_{\text{bus}}$ (CSC); N_{pq} , $n_a = N_{pq} + N_{pv}$; buses [PQ | PV | slack]
Ensure: col_ptrs , row_idx of $\mathbf{J}_{\text{red}}$; $pq_end[k]$, $active_end[k]$, $diag[k]$; block start offsets
1: **for** $k = 0, \dots, n_a - 1$ **do**
2: bisection rows of $\mathbf{Y}_{\text{bus}}^{:,k}$ at N_{pq} and n_a ; record $pq_end[k]$, $active_end[k]$, $diag[k]$
3: append rows $[0, n_a]$ of $\mathbf{Y}_{\text{bus}}^{:,k}$ to row_idx as θ_k column of $\mathbf{J}_{\text{red}}$; record J_{11} , J_{21} start offsets
4: **if** $k < N_{pq}$ **then** append same rows as $|V|_k$ column; record J_{12} , J_{22} start offsets
5: **end if**
6: seal column in col_ptrs
7: **end for**

Algorithm 2 FILL — in-place numeric assembly of $\mathbf{J}_{\text{red}}$

Require: $\mathbf{Y}_{\text{bus}}$, $\mathbf{V}$, $\hat{\mathbf{V}}$, $\mathbf{I}_{\text{bus}} = \mathbf{Y}_{\text{bus}}\mathbf{V}$; cache from Alg. 1
Ensure: value array of $\mathbf{J}_{\text{red}}$ overwritten in place
1: **for** $k = 0, \dots, N_{pq} - 1$ **do** ▷ PQ column: write θ_k and $|V|_k$
2: **for** $(i, k) \in \text{nz}(\mathbf{Y}_{\text{bus}})$, $i < n_a$ **do**
3: write $\mathcal{U}_\theta(i, k)$, $\mathcal{U}_v(i, k)$ (18) to J_{11} , J_{12} ; also J_{21} , J_{22} if $i < N_{pq}$
4: **end for**
5: add $(-Q_k, +P_k, P_k/|V_k|, Q_k/|V_k|)$ to diagonal entries (21)
6: **end for**
7: **for** $k = N_{pq}, \dots, n_a - 1$ **do** ▷ PV column: write θ_k only
8: **for** $(i, k) \in \text{nz}(\mathbf{Y}_{\text{bus}})$, $i < n_a$ **do**
9: write $\mathcal{U}_\theta(i, k)$ (18) to J_{11} ; also J_{21} if $i < N_{pq}$
10: **end for**
11: add $-Q_k$ to J_{11} diagonal entry (21)
12: **end for**

corollaries derived in Section III, the runtime completely isolates the PQ (Lines 1–5) and PV (Lines 6–10) loops, thereby eliminating all conditional branching inside the hot execution path. Because both the input layout of $\mathbf{Y}_{\text{bus}}$ and the destination offsets of $\mathbf{J}_{\text{red}}$ are mathematically locked, this element-wise fill operates in a single, direct pass, simultaneously streaming computed sensitivities into multiple distinct matrix memory starting points.

By ensuring contiguous memory access, this algorithm allows standard compilers to automatically inject SIMD vectorization and achieve maximum cache utilization. Furthermore, the total elimination of branch divergence and index arithmetic satisfies the core architectural prerequisites for GPU execution, making this numeric kernel natively suited for massively parallel acceleration.

The global integration of these modules is formalized in Algorithm 3. Rather than proposing a modified iterative framework, Algorithm 3 serves as a straightforward replacement for the standard sequence. By exposing a unified trait within *RustPower*, the offline symbolic pass (Algorithm 1) and the in-place numeric kernel (Algorithm 2) simply substitute

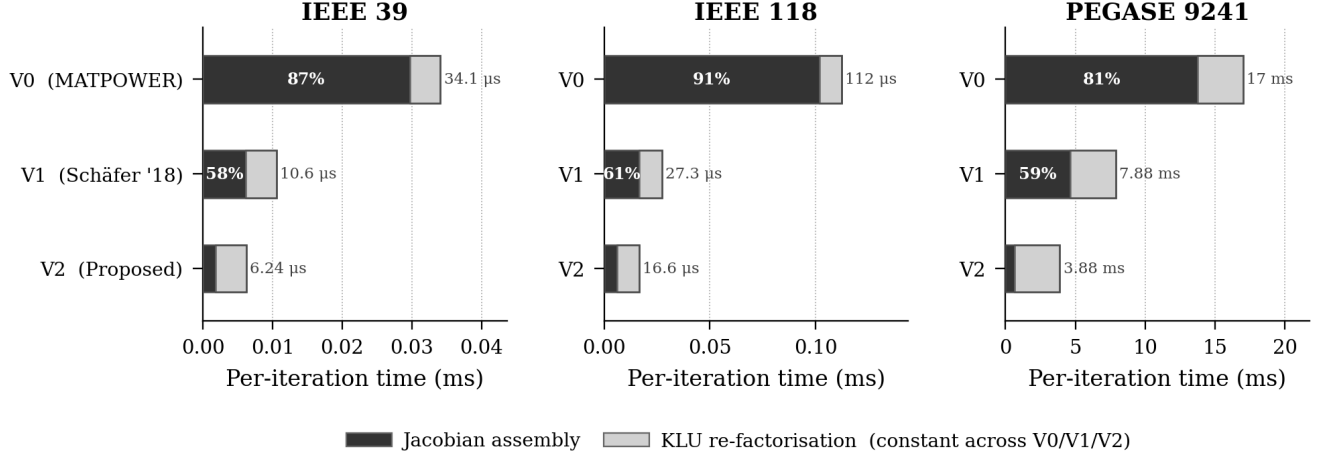


Fig. 3. Iteration cost breakdown on the test systems: assembly and factorisation of three methods in *RustPower*. The proposed method (V2) significantly reduces the assembly share, making factorisation the dominant cost.

Algorithm 3 Newton–Raphson with symbolic-cached assembly

Require: $Y_{\text{bus}}, S_{\text{bus}}^{\text{sp}}, V, N_{pv}, N_{pq}$, tolerance τ

- 1: $\text{cache} \leftarrow \text{Algorithm 1}$ $\triangleright$ symbolic phase, once
- 2: allocate the value array of J_{red} $\triangleright$ the only Jacobian allocation
- 3: **for** $\nu = 0, 1, 2, \dots$ **do**
- 4: $I_{\text{bus}} \leftarrow Y_{\text{bus}} V$; $F \leftarrow$ power mismatch (2)
- 5: **if** $\|F\| < \tau$ **then**
- 6: **return** V
- 7: **end if**
- 8: run Algorithm 2 $\triangleright$ overwrite J_{red} values in place
- 9: solve $J_{\text{red}} \Delta x = -F$ $\triangleright$ structure from the cache
- 10: $V \leftarrow$ voltage update from Δx
- 11: **end for**

the conventional assembly block, allowing the simulator to interface with interchangeable direct solvers like KLU.

V. CASE STUDIES

A. Experimental Setup

Performance was measured on an i7-10700K CPU at 4.7 GHz with 32 GB DDR4 memory. Three networks were tested: the IEEE 39-bus, the IEEE 118-bus [26], and the PEGASE 9241-bus [27] systems. On IEEE 39, transformers are modeled as lines to match pandapower for validation. All can be accessed from the *RustPower* repository [28].

We evaluate three paths compiled within the same binary, using the same solver instance:

- V0 — A direct port of MATPOWER. It computes complex matrices (8)–(9) using diagonal products and rebuilds the Jacobian via block stacking on every iteration.
- V1 — An optimized formulation. It uses symbolic permutation from *RustPower*, a tuned derivative function and cached buffers, but keeps the three-step assembly. It is similar to [16] but with the pre-permuted partition of PQ,PV nodes.

- V2 — The proposed method of Sections III–IV.

Results are averaged over 300 solves. We compare these paths against pandapower [14] and LightSim2Grid [15]. LightSim2Grid uses a C++ backend on a pre-built model. Both tools run native solves.

B. Accuracy

The three paths match to machine precision; the differences $\|v_{V0} - v_{V2}\|$ and $\|v_{V1} - v_{V2}\|$ remain below 10^{-13} . On all systems, the solver converges to the same solution as pandapower. Voltages and angles agree within the 1^{-6} tolerance.

C. Internal Ablation: Isolating the Symbolic Gain

Table II reports solve times. Transitioning from V0 to V1 yields a $1.9\times$ to $2.7\times$ speedup via memory tuning. Transitioning from V1 to V2 provides an additional $1.5\times$ to $1.8\times$ speedup. Because these totals include the same solve, they understate the assembly speedup.

Table III isolates the assembly step. Here, the symbolic assembly is $2.8\times$ to $7.2\times$ faster than the tuned path, and $16\times$ to $21\times$ faster than the V0 port. The speedup remains independent of the network size.

Profiling a single iteration reveals the runtime allocation (Fig. 3). In the standard port, assembly consumes 87%, 91%, and 81% of the iteration time. Tuning reduces this share to 58%, 61%, and 59%. The proposed method eliminates the structural bottleneck, dropping the assembly share to 29%, 36%, and 17%. Consequently, numeric factorization becomes the primary cost.

This speedup stems from structural simplification. Traditional assembly traverses nonzeros repeatedly to form intermediate matrices, sub-blocks, and the stacked Jacobian. The symbolic pass eliminates intermediates. By pre-mapping where each nonzero lands, the numeric phase reads the admittance matrix once and writes the Jacobian once.

TABLE II
ABLATION OF JACOBIAN ASSEMBLY: SOLVE TIME (MS) WITH SAME SOLVER.

System	V0	V1	V2	V0/V2	V1/V2
IEEE 39	0.159	0.060	0.039	4.1×	1.5×
IEEE 118	0.496	0.181	0.099	5.0×	1.8×
PEGASE 9241	85.8	44.3	30.0	2.9×	1.5×

TABLE III
ISOLATED ASSEMBLY STEP: ASSEMBLY TIME (μ s) EXCLUDING SOLVE.

System	V0	V1	V2	V0/V2	V1/V2
IEEE 39	29.7	6.19	1.84	16.1×	3.4×
IEEE 118	102	16.7	6.03	16.9×	2.8×
PEGASE 9241	13745	4635	641	21.4×	7.2×

D. Comparison with Established Tools

Table IV compares the speed-up with open-source SOTA solvers. The proposed method V2 in `RustPower` is 3.1×

 and 3.5× faster than C++ `LightSim2Grid` on the IEEE 39 and 118 systems, and 1.7× faster on PEGASE 9241. This performance margin scales with the network size, which confirms that the gain represents saved assembly operations rather than framework overhead. On PEGASE 9241, factorization dominates the execution, which compresses the relative margin, but the speedup persists. All `pandapower` results use version 3.x. Compared to version 2.x, v3.x exhibits significantly higher per-solve overhead on small networks (IEEE 39 was 3–10 ms on v2.x) but lower overhead on large ones (PEGASE 9241 was approximately 1.3 s on v2.x); the benchmarks reflect current release behavior.

Both reference tools fix the sparse structure to reuse the symbolic analysis. While C++ templates enable compile-time matrix expression optimization, `LightSim2Grid`'s runtime still performs three sequential steps inside the hot loop, whereas the proposed method eliminates them entirely.

Notably, even the V1 optimization outperforms `LightSim2Grid` due to the symbolic pre-permuted $\mathbf{Y}_{\text{bus}}$ partitions designed in `RustPower`. This confirms that Theorem 1's insight on symbolic permutation provides an immediate speedup even for the traditional three-step assembly, further underscoring its profound significance in overcoming modern hardware memory bottlenecks.

Memory measurements show that `RustPower` limits peak allocation to 34 MB on PEGASE 9241, which is a 15-fold reduction compared to over 500 MB for `pandapower`. This reduction results from the proposed efficient allocation-free method as well as superior memory management in Rust.

VI. CONCLUSION

This paper delivers three contributions for steady-state power-flow analysis. First, we mathematically prove the permutation equivariance of complex sensitivity, verifying that symbolic permutation pre-applied to the admittance matrix eliminates runtime projections. Second, we develop a generic algorithm that couples symbolic permutation with branchless filling to assemble the global Jacobian in a single pass.

TABLE IV
CORE NEWTON-RAPHSON SOLVE TIME (MS).

System	PP	LS2G	<code>RustPower</code>	vs LS2G	vs PP
IEEE 39	38.9	0.12	0.039	3.1×	~1000×
IEEE 118	42.8	0.35	0.099	3.5×	~430×
PEGASE 9241	145.5	51.2	30.0	1.7×	~4.9×

PP = `pandapower`; LS2G = `LightSim2Grid`.

Third, we realize this framework in the `RustPower` solver. Benchmarks demonstrate significant acceleration of power-flow computations, delivering a 1.7×

 to 3.5× speedup over the state-of-the-art open-source solver `LightSim2Grid`, while reducing assembly overhead by 7.2–21× compared to baselines without altering the numeric results or the convergence.

The proposed methodology, the generic algorithm, and the `RustPower` toolchain establish a new foundation for modern power system analysis. The new method has potential for faster optimal power flow, pre-conditioning, and continuous Newton power-flow solvers. It may also be applicable to transient stability simulation, where streaming logic provides optimal performance on multi-GPU architectures [29] and even FPGAs [30]. Ultimately, while artificial intelligence increases the demand for advanced computation, the proposed work helps bridge the widening gap between evolving computing paradigms and traditional analysis tools for modern power systems.

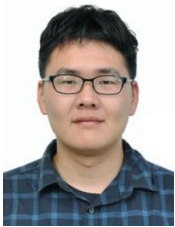
REFERENCES

- [1] W. F. Tinney and C. E. Hart, "Power flow solution by Newton's method," *IEEE Trans. Power App. Syst.*, vol. PAS-86, no. 11, pp. 1449–1460, 1967.
- [2] W. F. Tinney and J. W. Walker, "Direct solutions of sparse network equations by optimally ordered triangular factorization," *Proc. IEEE*, vol. 55, no. 11, pp. 1801–1809, 1967.
- [3] T. A. Davis and E. P. Natarajan, "Algorithm 907: KLU, a direct sparse solver for circuit simulation problems," *ACM Trans. Math. Softw.*, vol. 37, no. 3, pp. 36:1–36:17, 2010.
- [4] R. Idema, D. J. P. Lahaye, C. Vuik, and L. van der Sluis, "Scalable Newton-Krylov solver for very large power flow problems," *IEEE Trans. Power Syst.*, vol. 27, no. 4, pp. 2248–2257, 2012.
- [5] E. F. Yetkin and O. Ceylan, "Recycling Newton-Krylov algorithm for efficient solution of large scale power systems," *Int. J. Electr. Power Energy Syst.*, vol. 144, p. 108559, 2023.
- [6] M. Iwamoto and Y. Tamura, "A load flow calculation method for ill-conditioned power systems," *IEEE Trans. Power App. Syst.*, vol. PAS-100, no. 4, pp. 1736–1743, 1981.
- [7] P. J. Lagace, M. H. Vuong, and I. Kamwa, "Improving power flow convergence by Newton-Raphson with a Levenberg-Marquardt method," in *Proc. IEEE Power Energy Soc. Gen. Meeting*, 2008, pp. 1–6.
- [8] R. Pires, L. Mili, and G. Chagas, "Robust complex-valued Levenberg-Marquardt algorithm as applied to power flow analysis," *Int. J. Electr. Power Energy Syst.*, vol. 113, pp. 383–392, 2019.
- [9] F. Milano, "An open source power system analysis toolbox," *IEEE Trans. Power Syst.*, vol. 20, no. 3, pp. 1199–1206, 2005.
- [10] R. Yu, W. Gu, Y. Xu, S. Lu, and S. Zhang, "Semi-implicit continuous newton method for power flow analysis," *CSEE J. Power Energy Syst.*, vol. 11, no. 4, pp. 1957–1961, 2025.
- [11] A. R. Bergen and V. Vittal, *Power Systems Analysis*, 2nd ed. Prentice Hall, 2000.
- [12] R. D. Zimmerman, C. E. Murillo-Sanchez, and R. J. Thomas, "MATPOWER: Steady-state operations, planning, and analysis tools for power systems research and education," *IEEE Trans. Power Syst.*, vol. 26, no. 1, pp. 12–19, 2011.
- [13] R. Lincoln, "PYPOWER: A Python port of MATPOWER," <https://pypi.python.org/pypi/PYPOWER>, 2015.

- [14] L. Thurner, A. Scheidler, F. Schäfer, J. Menke, J. Dollichon, F. Meier, S. Meinecke, and M. Braun, “pandapower — An open-source Python tool for convenient modeling, analysis, and optimization of electric power systems,” *IEEE Trans. Power Syst.*, vol. 33, no. 6, pp. 6510–6521, 2018.
- [15] B. Donnot, “LightSim2Grid: A C++ backend targeting the Grid2Op platform for power-system simulation,” <https://github.com/BDonnot/lightsim2grid>, 2020.
- [16] F. Schäfer and M. Braun, “An efficient open-source implementation to compute the jacobian matrix for the Newton-Raphson power flow algorithm,” in *Proc. IEEE PES Innov. Smart Grid Technol. Conf. Europe (ISGT-Europe)*, 2018, pp. 1–6, arXiv:1804.06742.
- [17] Z. Wang, S. W. von Berg, and M. Braun, “Fast parallel Newton–Raphson power flow solver for large number of system calculations with CPU and GPU,” *Sustain. Energy Grids Networks*, vol. 27, p. 100483, 2021.
- [18] G. Zhou, Y. Feng, R. Bo, L. Chien, X. Zhang, Y. Lang, Y. Jia, and Z. Chen, “GPU-accelerated batch-ACPF solution for N-1 static security analysis,” *IEEE Trans. Smart Grid*, vol. 8, no. 3, pp. 1406–1416, 2017.
- [19] G. Zhou, R. Bo, L. Chien, X. Zhang, S. Yang, and D. Su, “GPU-accelerated algorithm for online probabilistic power flow,” *IEEE Trans. Power Syst.*, vol. 33, no. 1, pp. 1132–1135, 2018.
- [20] M. Ryu, G. Byeon, and K. Kim, “A GPU-accelerated distributed algorithm for optimal power flow in distribution systems,” in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*, 2025, pp. 702–711.
- [21] S. Klabnik and C. Nichols, *The Rust Programming Language*. No Starch Press, 2019. [Online]. Available: <https://doc.rust-lang.org/book/>
- [22] Bevy Contributors, “Bevy: A data-driven game engine built in Rust,” <https://bevyengine.org>, 2020.
- [23] R. Nystrom, “Data locality (game programming patterns),” in *Game Programming Patterns*. Genever Benning, 2014. [Online]. Available: <https://gameprogrammingpatterns.com/data-locality.html>
- [24] T. Cheng, T. Duan, and V. Dinavahi, “ECS-Grid: Data-oriented real-time simulation platform for cyber-physical power systems,” *IEEE Trans. Ind. Informat.*, vol. 19, no. 11, pp. 11 128–11 138, 2023.
- [25] U. Drepper, “What every programmer should know about memory,” Red Hat, Inc., Tech. Rep., 2007. [Online]. Available: <https://www.akkadia.org/drepper/cpumemory.pdf>
- [26] University of Washington, “IEEE 118-bus test case,” Power Systems Test Case Archive, University of Washington, Tech. Rep., 1993. [Online]. Available: https://labs.ece.uw.edu/pstca/pf118/pg_tca118bus.htm
- [27] S. Fliscounakis, P. Panciatici, F. Capitanescu, and L. Wehenkel, “Contingency ranking with respect to overloads in very large power systems taking into account uncertainty, preventive, and corrective actions,” *IEEE Trans. Power Syst.*, vol. 28, no. 4, pp. 4909–4917, 2013.
- [28] T. Cheng, “RustPower: A data-oriented open-source power-flow solver in Rust,” <https://github.com/chengts95/rustpower>, 2024.
- [29] V. Jalili-Marandi, Z. Zhou, and V. Dinavahi, “Large-scale transient stability simulation of electrical power systems on multiple GPUs,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 23, no. 7, pp. 1255–1265, 2012.
- [30] S. Cao, N. Lin, and V. Dinavahi, “Faster-than-real-time hardware emulation of extensive contingencies for dynamic security analysis of large-scale integrated AC/DC grid,” *IEEE Trans. Power Syst.*, vol. 38, no. 1, pp. 861–871, 2023.



Venkata Dinavahi (Fellow, IEEE) received the B.Eng. degree in electrical engineering from the Visvesvaraya National Institute of Technology, Nagpur, India, the M.Tech. degree in electrical engineering from the Indian Institute of Technology, Kanpur, India, and the Ph.D. degree in electrical and computer engineering from the University of Toronto, Ontario, Canada. He is currently a Professor with the Department of Electrical and Computer Engineering, University of Alberta, Edmonton, Canada. His research interests include real-time simulation of multi-domain sustainable energy systems, electromagnetic transients, device-level modeling, artificial intelligence, large-scale systems, and parallel and distributed computing. Dr. Dinavahi is a Fellow of the Engineering Institute of Canada (EIC), the Asia-Pacific Artificial Intelligence Association (AIAA), the International Artificial Intelligence Industry Alliance (AIIA). He is an honorary member of the National Academy of Artificial Intelligence (NAAI), and a Professional Engineer in the Province of Alberta, Canada.



Tianshi Cheng (S'19-M'24) received the B.Eng. degree in electrical engineering and automation from Southeast University, China, in 2017, and the Ph.D. degree in electrical and computer engineering from the University of Alberta, Canada, in 2024. From 2017 to 2018, he was a substation automation engineer at NARI Group Corporation (State Grid Electric Power Research Institute), China. He is currently working as a Simulation Specialist at RTDS Technologies Inc., Winnipeg, MB, Canada. His research interests include electromagnetic transient simulation, transient stability analysis, steady-state analysis, real-time hardware-in-the-loop simulation, parallel processing, microgrids, and power electronics.

tion, transient stability analysis, steady-state analysis, real-time hardware-in-the-loop simulation, parallel processing, microgrids, and power electronics.